AVR 기반
자율주행차 프로젝트

발표자 박상현

010-2791-2479

psh010208@naver.com

목 차

01 프로젝트 개요

02 외관 사진

03 회로도 / BOM Table

04 FSM

05 동작 구현

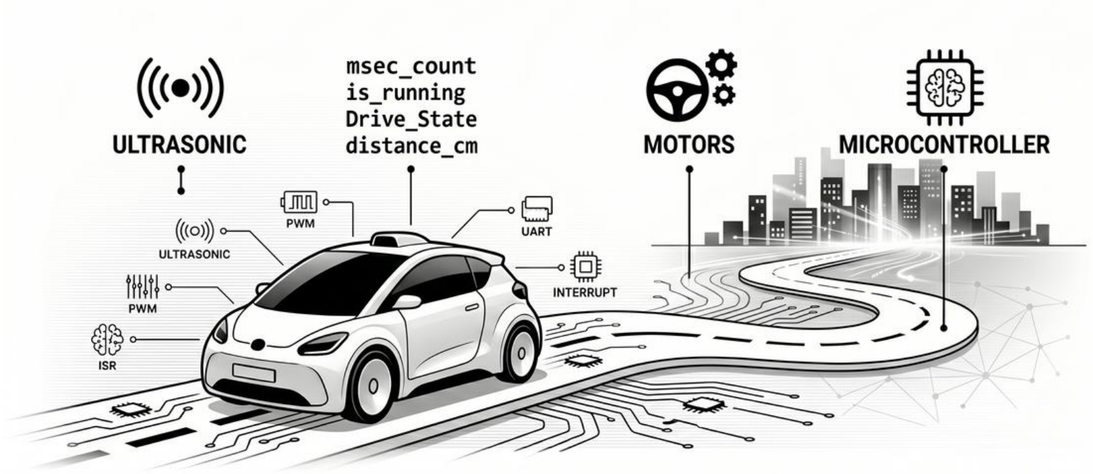
06 코드 설명

07 요소 기술

08 결론

프로젝트 개요

초음파센서 AVR 기반 자율주행차 프로젝트



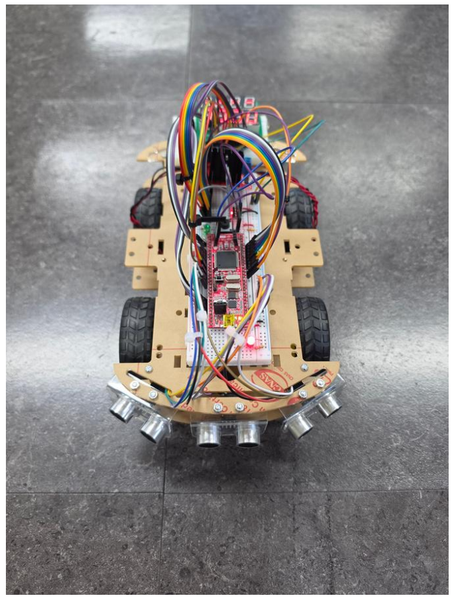
프로젝트 배경 및 목적

본 프로젝트는 8비트 AVR 마이크로컨트롤러인 ATmega128을 활용하여 실시간 센서 데이터 처리와 모터 드라이버 제어가 결합된 스마트 자율주행 모형차를 설계하고 구현하는 것을 목적으로 함

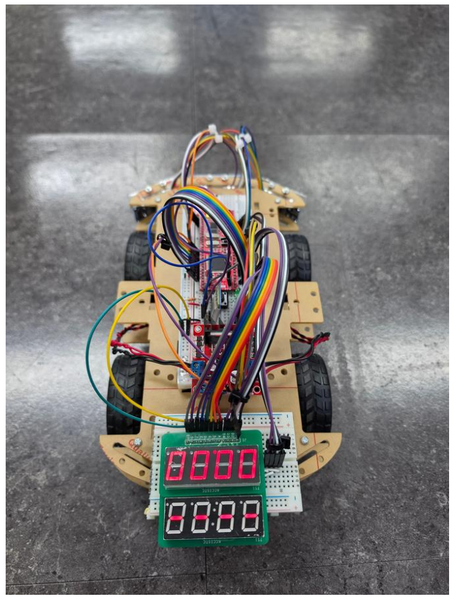
단순한 모형차 구동을 넘어, 시스템에 적용된 핵심 임베디드 SW 기술들을 하드웨어 레벨에서 명확히 이해하고 제어 능력을 체득하는 데 중점을 둠

#자율주행 #AVR #ATmega128

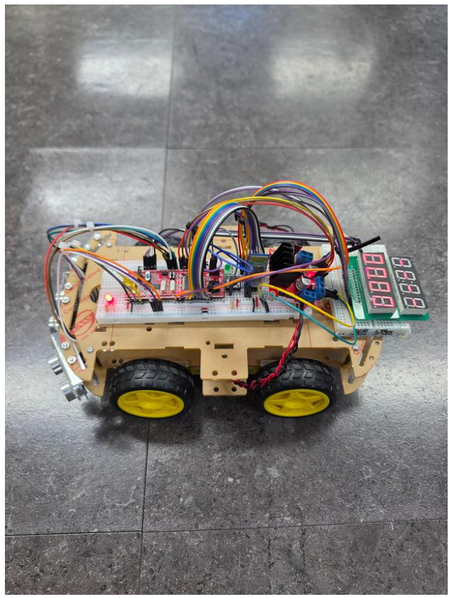
외관 사진



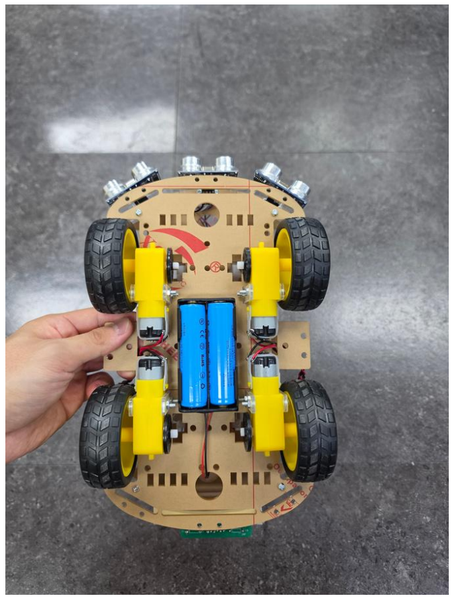
전면 사진



후면 사진



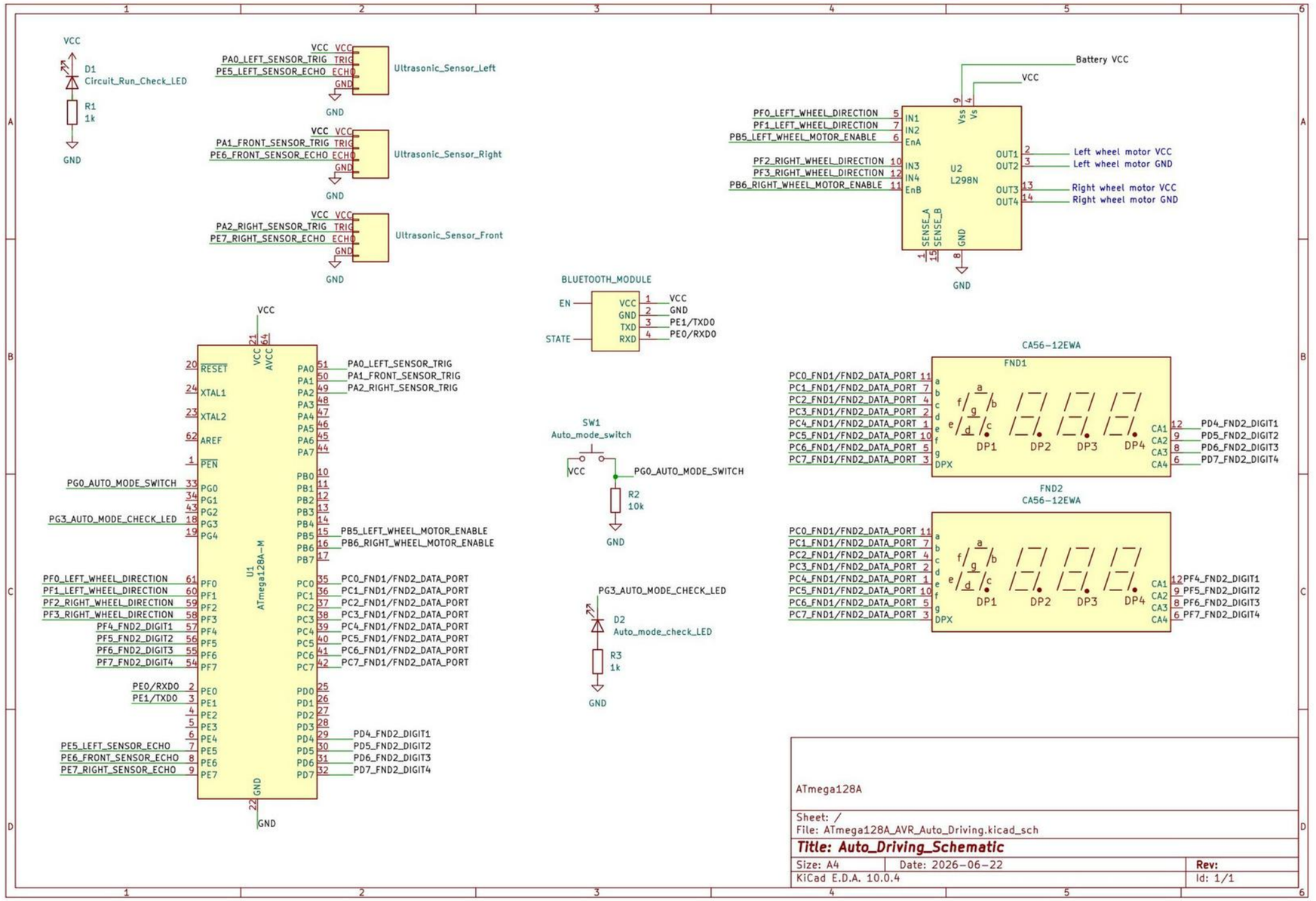
측면 사진



하부 사진

회로도

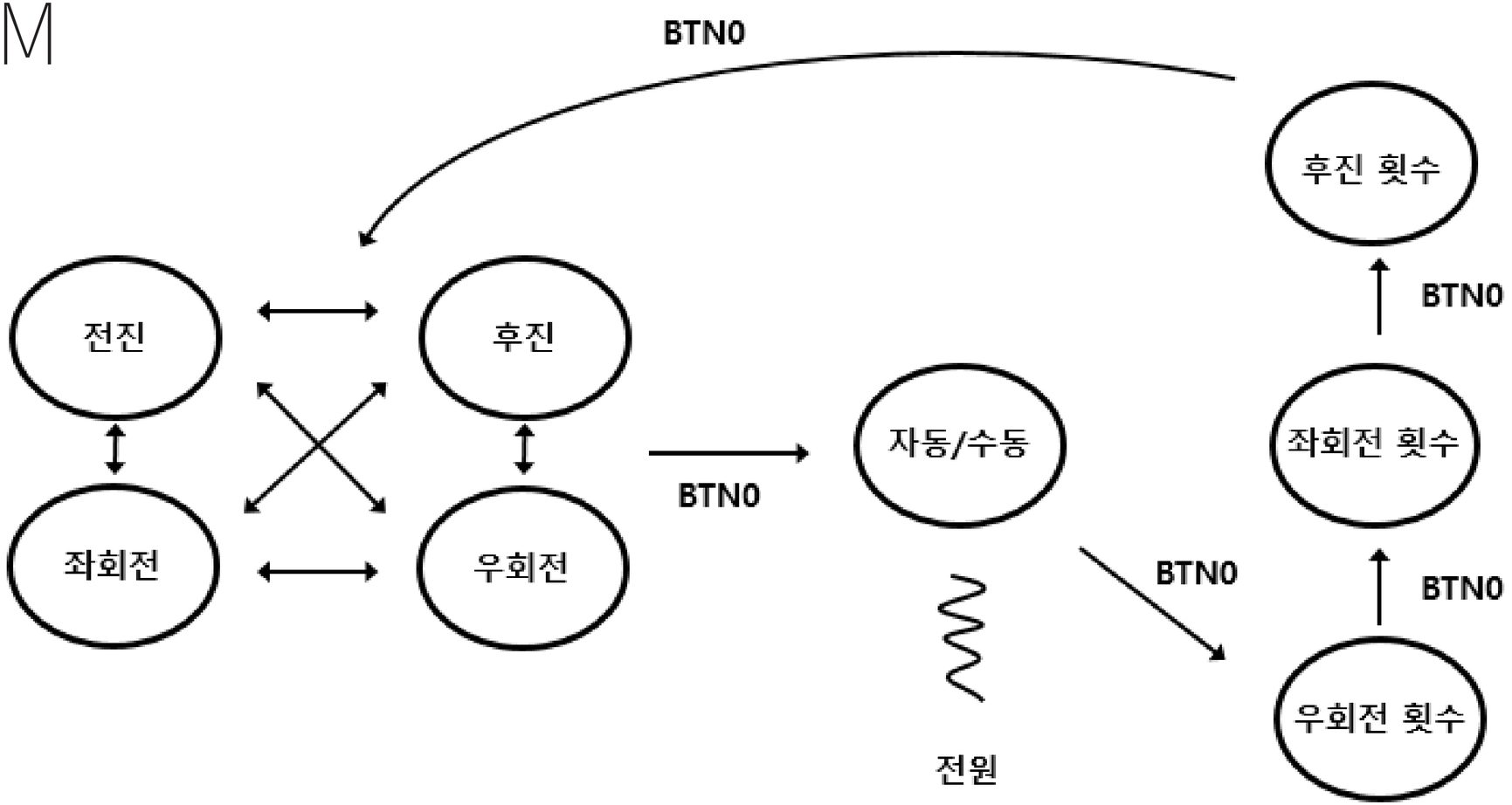
KiCad 10.0



BOM Table

No.	부품명	부품	용도	수
1	ATmega128A	ATmega128A	메인 컨트롤러	1
2	모터 드라이버	L298N	모터 구동 및 방향 제어	1
3	DC 모터	-	자율주행차 구동원	4
4	바퀴(Wheel)	-	자율주행차 주행체	4
5	초음파 센서	HC-SR04	장애물 거리 계산	3
6	블루투스 모듈	HC-06 (HC-05)	수동 모드 명령 송수신	1
7	택트 스위치	6x6 Tact Switch	모드 변경	1
8	FND	WCN4-0056B7-A11	주행 시간, 주행 방향, 주행 기록 표시	2
9	LED (적색)	5mm	전원 공급 표시	1
10	LED (녹색)	5mm	자율주행 모드 표시	1
11	330Ω 저항	-	전원 LED, FND 저항	17
12	1kΩ 저항	-	자율주행 LED 저항	1
13	10kΩ 저항	-	스위치 저항	1
14	초음파 센서 고정 홀더	-	초음파 센서 고정 기구	3
15	자동차 아크릴 판	-	자율주행차 프레임	2
16	브레드보드	-	브레드보드	2
17	배터리 홀더	-	배터리 고정 기구	1
18	볼트, 너트	-	구조 결합 부품	20
20	점퍼 와이어	-	전기적 신호 연결 도선	47

FSM


BTN0
전진
후진
좌회전
우회전
BTN0
자동/수동
전원
BTN0
후진 횟수
BTN0
좌회전 횟수
BTN0
우회전 횟수

동작 구현

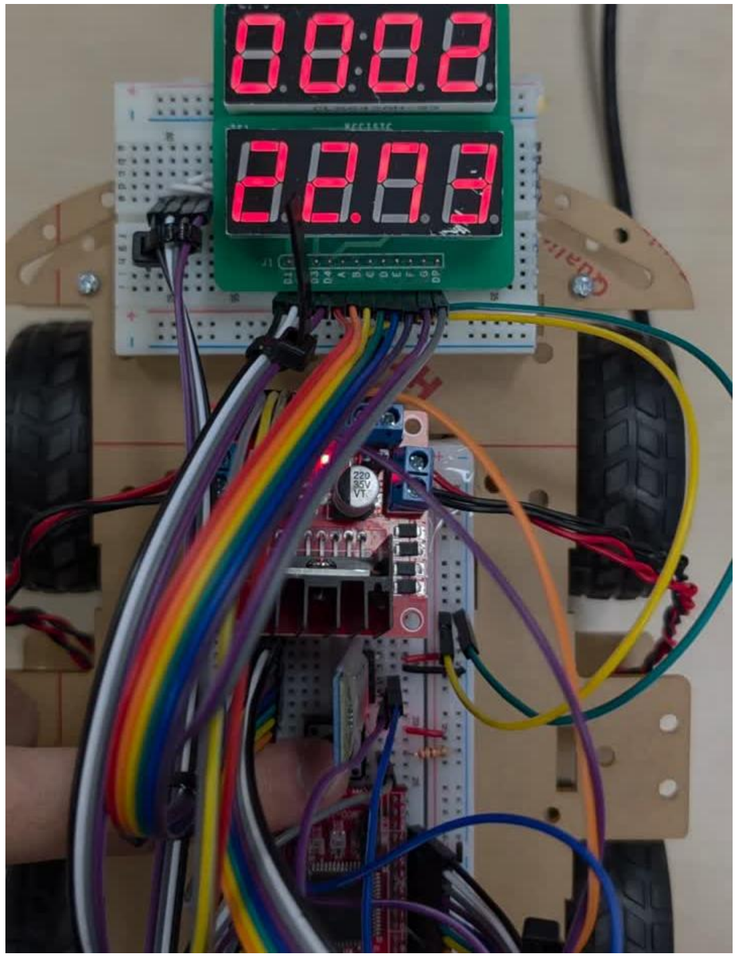
버튼 조작

- 주행 종료 이후 버튼 조작에 의해 FND에 결과 조회 가능
- FND1: 총 주행 시간
- FND2: 후진 횟수 / 좌회전 횟수 / 우회전 횟수
- FND2:: 정시 상태인 경우 ‘- - - -’

자율 주행

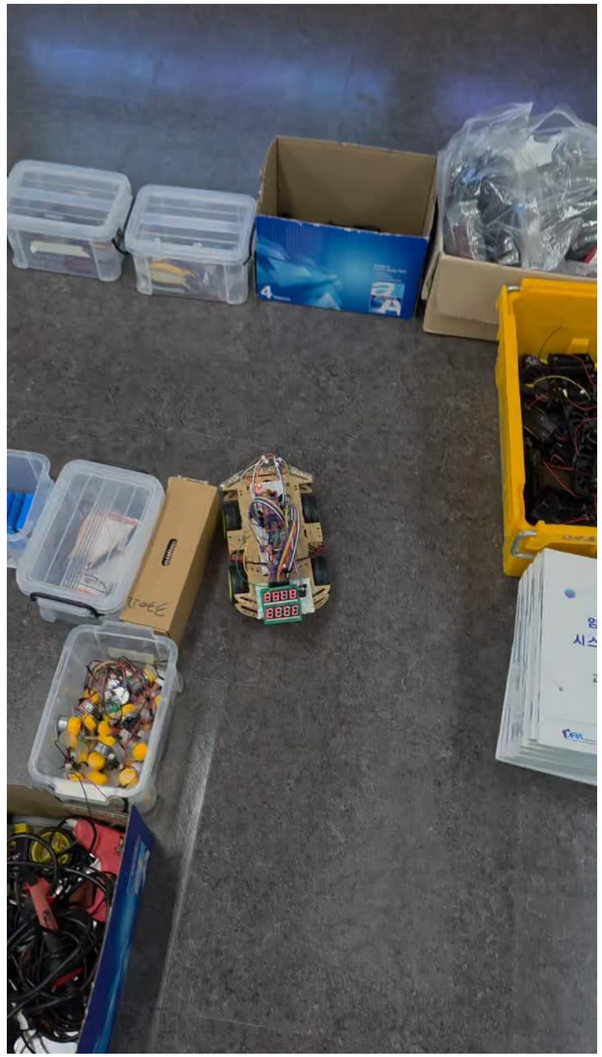
- 자율주행 모드 동작 시, FND에 현 상태 확인 가능
- FND1: 현 주행 시간
- FND2: 진행 방향
- 전진: 시계 방향 회전, 후진: 반시계 방향 회전
- 좌회전: ‘[[[[’, 우회전: ‘]]]]’

동작 구현



버튼 조작 영상

동작 구현



자율 주행 동작 영상

코드 설명

초음파 센서 거리 계측 ISR

```
    // 각 센서별 거리 및 데이터 보관 배열 [0]:좌, [1]:정면, [2]:우
    volatile int distance_cm[3] = {0, 0, 0};


ISR(INT4_vect)
{
    if (ECHO_PORT & (1 << ECHO_LEFT)) // 상승 에지
    {
        TCNT3 = 0;
    }
    else // 하강 에지
    {
        uint32_t ultrasonic_distance = TCNT3 * 1000000.0 * 1024 / F_CPU;
        distance_cm[LEFT_SENSOR] = ultrasonic_distance / 58;
        falling_edge_flag = 1;
    }
}
```

에지(Edge) 판별을 통한 시간 측정

- 상승 에지 감지 시 (초음파가 발사된 후 에코 핀이 High가 된 시점)
- 타이머 카운터 레지스터(TCTN)를 초기화하여 계측 시작

- 하강 에지 감지 시 (반사된 초음파가 수신되어 에코 핀이 Low로 떨어진 시점)
- TCNTN 값을 읽어 왕복 소요 시간 계산

거리 환산 및 동기화 플래그 처리

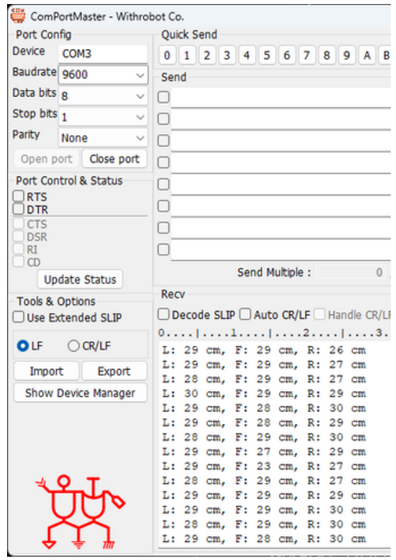
- 1024 분주비와 F_CPU(16MHz)를 반영해 왕복 시간을 마이크로초 단위로 변환
- 초음파 속도 상수를 반영한 음속 공식인 '/ 58' 연산을 거쳐 cm 단위 거리 산출

코드 설명

실시간 3방향 초음파 순환 제어 스케줄러

```
void ultrasonic_processing(void)
{
    if (falling_edge_flag == 1)
    {
        falling_edge_flag = 0;

        if (current_sensor == LEFT_SENSOR)
        {
            sprintf(scm_buf, "L: %d cm, ", distance_cm[LEFT_SENSOR]);
            printf("%s", scm_buf);
            current_sensor = FRONT_SENSOR; // 다음은 정면
            make_trigger(TRIG_FRONT);
        }
        else if (current_sensor == FRONT_SENSOR)
        {
            sprintf(scm_buf, "F: %d cm, ", distance_cm[FRONT_SENSOR]);
            printf("%s", scm_buf);
            current_sensor = RIGHT_SENSOR; // 다음은 우측
            make_trigger(TRIG_RIGHT);
        }
        else if (current_sensor == RIGHT_SENSOR)
        {
            sprintf(scm_buf, "R: %d cm\n", distance_cm[RIGHT_SENSOR]);
            printf("%s", scm_buf);
            current_sensor = LEFT_SENSOR;  // 다시 좌측으로 턴 루프
            make_trigger(TRIG_LEFT);
        }
    }
}
```



시분할 순환 제어 (Time-Division Round-Robin)

- 3개의 초음파 센서를 동시에 쏘면 주파수 간섭(유령 에코)이 발생하므로, 좌 ➔ 정면 ➔ 우 순서로 한 번에 하나의 센서만 트리거하는 시분할 방식을 채택
- 하나의 센서 측정이 완벽히 끝나 하강 에지 인터럽트 (falling_edge_flag == 1)가 발생한 순간에만 다음 센서의 트리거를 가동

상태 머신 기반의 차례(Turn) 관리

- current_sensor 상태 변수를 모니터링하여 현재 종료된 센서의 데이터를 ComPortMaster로 (printf)로 출력
- 측정 완료 ➔ 다음 상태로 전환 후 해당 트리거 발생

코드 설명

최적 공간 탐색 자율주행 알고리즘

```
void auto_mode(void)
{
    distance_check();  // 초음파 센서 실시간 거리 갱신

    int left_dis  = distance_cm[0];
    int front_dis = distance_cm[1];
    int right_dis = distance_cm[2];

    int max_dis = front_dis; // 정면을 가장 멀다고 가정

    if (left_dis > max_dis)
    {
        max_dis = left_dis;  // 왼쪽이 더 멀다면 교체
    }
    if (right_dis > max_dis)
    {
        max_dis = right_dis; // 오른쪽이 더 멀다면 교체
    }

    // 만약 삼면이 다 벽에 막혔으면 후진
    if (max_dis <= OBSTACLE_DISTANCE)
    {
        backward(SPEED);
    }
    // 가장 넓은 최적의 방향을 선택해서 이동
    else
    {
        if (max_dis == front_dis)
        {
            forward(SPEED); // 정면이 가장 넓음 -> 직진
        }
        else if (max_dis == left_dis)
        {
            turn_left(TURN_SPEED); // 왼쪽이 가장 넓음 -> 좌회전
        }
        else if (max_dis == right_dis)
        {
            turn_right(TURN_SPEED); // 오른쪽이 가장 넓음 -> 우회전
        }
    }
}
```

탐욕적 경로 선택 (Greedy Path Selection)

- 단순히 전방 장애물 유무만 판단하는 것이 아니라, 좌·정면·우 세 방향의 데이터를 실시간으로 상호 비교
- 비교 연산을 통해 세 방향 중 가장 먼 거리(즉, 장애물이 가장 없는 가장 넓은 공간)를 산출하여 주행 방향으로 결정

워크플로우

- 데이터 동기화: distance_check()를 호출하여 좌/정면/우측의 최신 거리 데이터를 확보
- 최적값 비교 판정: max_dis 변수를 활용한 비교 로직을 통해 가장 확보된 거리가 긴 방향을 탐색
- 주행 기동 제어: 탐색된 최적 방향과 모터 제어 함수를 매핑하여 기동

코드 설명

단일 버튼 기반 다중 제어 순환 인터페이스

```c
typedef enum
{
    DISPLAY_DRIVE_STATE, // 주행 중 방향 표시
    DISPLAY_BACK_COUNT,  // 후진 횟수 표시
    DISPLAY_LEFT_COUNT,  // 좌회전 횟수 표시
    DISPLAY_RIGHT_COUNT  // 우회전 횟수 표시
} FndDisplayMode;
```

```c
// 버튼 조작 시 외부에서 호출해줄 함수
void fnd_change_display_mode(void)
{
    if (cur_drive_state == STOP)
    {
        current_fnd_mode = (current_fnd_mode + 1) % 4;

        if (current_fnd_mode == DISPLAY_DRIVE_STATE)
        {
            type_mode = (type_mode == MANAUL_MODE) ? AUTO_MODE : MANAUL_MODE;
            led_auto_mode(type_mode);
        }
    }
}
```

자원 제한 극복을 위한 멀티플렉싱(Multiplexing)

- 설계 배경: 물리적 버튼이 BUTTON0 하나뿐인 하드웨어 환경 극복 필요
- 동작 원리: 차량이 완전 정지(STOP) 상태일 때, 버튼 입력 하나에 [데이터 조회]와 [시스템 모드 스위칭]이라는 두 가지 역할을 시분할 매핑

모드 자동 전환 메커니즘

- 버튼을 누를 때마다 current_fnd_mode 상태가 체인 형태로 순차 전환됨
- DISPLAY_DRIVE_STATE인 경우 차량의 주행 모드를 토글함

코드 설명

스톱워치 디스플레이

```
void fnd_stop_watch(void)
{
    static int digit_select = 0; // 자리수 선택

    fnd_clear();

    switch (digit_select)
    {
        case 0: // 1 단위
            FND_DIGIT_PORT_F &= ~(1 << FND_DIGIT_F4);
            FND_DATA_PORT = fnd_font[(ms_count % 100) / 10];
            break;
        case 1: // 10 단위
            FND_DIGIT_PORT_F &= ~(1 << FND_DIGIT_F5);
            FND_DATA_PORT = fnd_font[ms_count / 100];
            break;
        case 2: // 100 단위
            FND_DIGIT_PORT_F &= ~(1 << FND_DIGIT_F6);
            FND_DATA_PORT = fnd_font[sec_count % 10] | fnd_font[10];
            break;
        case 3: // 1000 단위
            FND_DIGIT_PORT_F &= ~(1 << FND_DIGIT_F7);
            FND_DATA_PORT = fnd_font[(sec_count / 10) % 6];
            break;
    }

    digit_select = (digit_select + 1) % 4; // 다음 표시할 자리수
}
```

다이내믹 스캐닝(Dynamic Scanning) 시분할 제어

- 제어 방식: 4자리의 FND를 제어하려면 32개의 범용 입출력 핀이 필요하지만, digit_select 변수를 통해 한 번에 한 자리씩 번갈아 켜는 시분할 구동 방식을 채택
- 잔상 효과: 인간의 착시 현상(잔상 효과)을 이용해 사용자 눈에는 4자리 숫자가 동시에 켜져 있는 것처럼 보이게 만듦
- 초기화 작업: 자리가 바뀔 때 이전 잔상이 남아 글자가 겹치는 고스트 현상을 방지하기 위해, DATA_PORT와 DIGIT_PORT를 초기화(Clear)한 후 동작함

코드 설명

주행 상태별 FND 그래픽 및 애니메이션 출력

```
// 주행 중 진행 방향 및 애니메이션 표시 함수
void fnd_drive_direction(void)
{
    static int digit_select = 0;

    fnd_clear();

    fnd_select_digit(digit_select);

    // 주행 상태 그래픽 출력
    switch (cur_drive_state)
    {
        case FORWARD:  FND_DATA_PORT = fnd_circle_anim_right[anim_index]; break;
        case BACKWARD: FND_DATA_PORT = fnd_circle_anim_left[anim_index]; break;
        case LEFT:     FND_DATA_PORT = 0x0F; break; // [ [ [ [
        case RIGHT:    FND_DATA_PORT = 0x39; break; // ] ] ] ]
        default:       FND_DATA_PORT = 0x40; break; // - - - -
    }

    digit_select = (digit_select + 1) % 4;
}
```

실시간 주행 상태 시각화

- 전진 (FORWARD): 시계 방향으로 회전하는 애니메이션 배열을 anim_index 주기에 따라 순차 출력
- 후진 (BACKWARD): 반시계 방향으로 회전하는 애니메이션 배열 출력
- 좌회전 (LEFT): 모든 자리에 왼쪽 방향을 가리키는 ‘ [’ 고정 출력
- 우회전 (RIGHT): 모든 자리에 오른쪽 방향을 가리키는 ‘] ’ 고정 출력
- 예외 상태 (default): 지정되지 않은 예외 상태 방지를 위해 기본 정지 ‘ - ’ 상태를 출력

코드 설명

주행 기록 조회

```
// 주행 종료(STOP) 후 버튼 조작에 따른 기록 조회 함수
void fnd_record_display(void)
{
    static int digit_select = 0;

    fnd_clear();

    fnd_select_digit(digit_select);

    uint32_t target_count = 0;

    // 버튼 조회 모드 판별
    switch (current_fnd_mode)
    {
        case DISPLAY_BACK_COUNT:  target_count = back_count;  break;
        case DISPLAY_LEFT_COUNT:  target_count = left_count;  break;
        case DISPLAY_RIGHT_COUNT: target_count = right_count; break;
        case DISPLAY_DRIVE_STATE:
        default:                  target_count = 9999;        break;
    }

    if (target_count == 9999)
    {
        FND_DATA_PORT = 0x40; // 기본 정지 상태 표시 (- - - -)
    }
    else
    {
        // 10진수 자릿수 파싱 분할 출력
        switch (digit_select)
        {
            case 0: FND_DATA_PORT = fnd_font[(target_count / 1000) % 10]; break;
            case 1: FND_DATA_PORT = fnd_font[(target_count / 100) % 10];  break;
            case 2: FND_DATA_PORT = fnd_font[(target_count / 10) % 10];   break;
            case 3: FND_DATA_PORT = fnd_font[target_count % 10];          break;
        }
    }

    digit_select = (digit_select + 1) % 4;
}
```

상태별 출력 데이터 매핑 (Data Mapping)

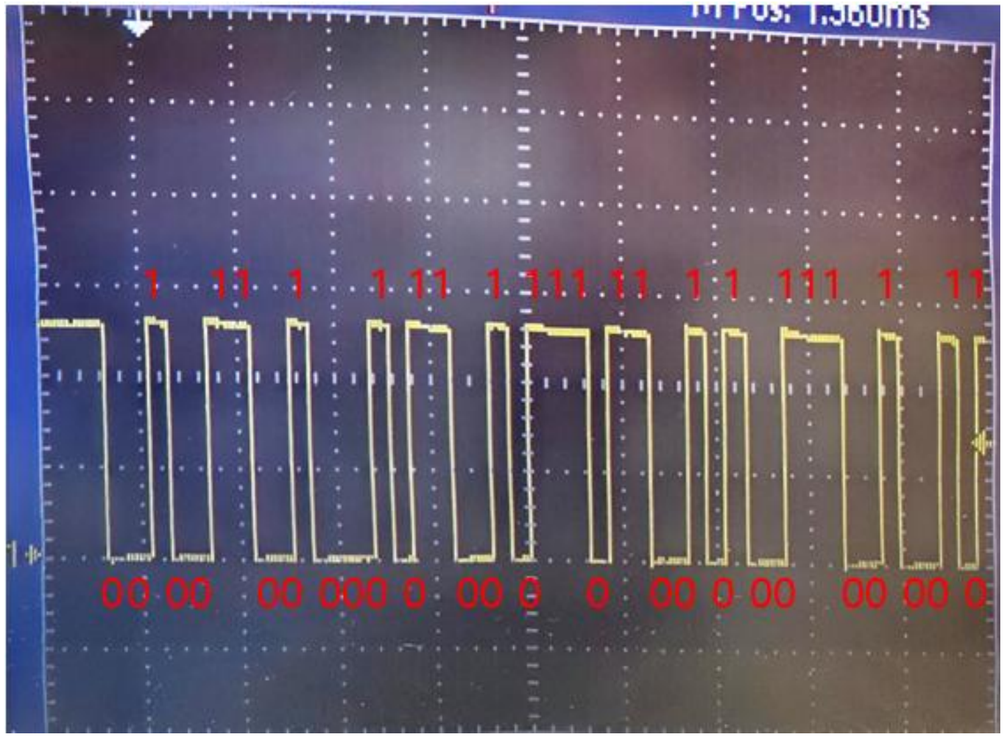
- 차량이 정지 상태일 때, 현재 FND 조회 상태에 따라 어떤 데이터를 출력할지 결정
- 각각의 전역 카운트 변수 값을 출력 타깃 변수에 대입

주행 데이터 출력

- 조회할 주행 데이터를, 자릿수에 맞춰 천의 자리부터 일의 자리까지 쪼갠 후 출력

UART 통신

요소 기술



1) 01001100 -> 0b00110010 -> 2

2) 00101100 -> 0b00110100 -> 4

3) 11101100 -> 0b00110111 -> 7

4) 10011100 -> 0b00111001 -> 9

5) 01010000 -> 0b00001010 -> A

직렬(Serial) 통신

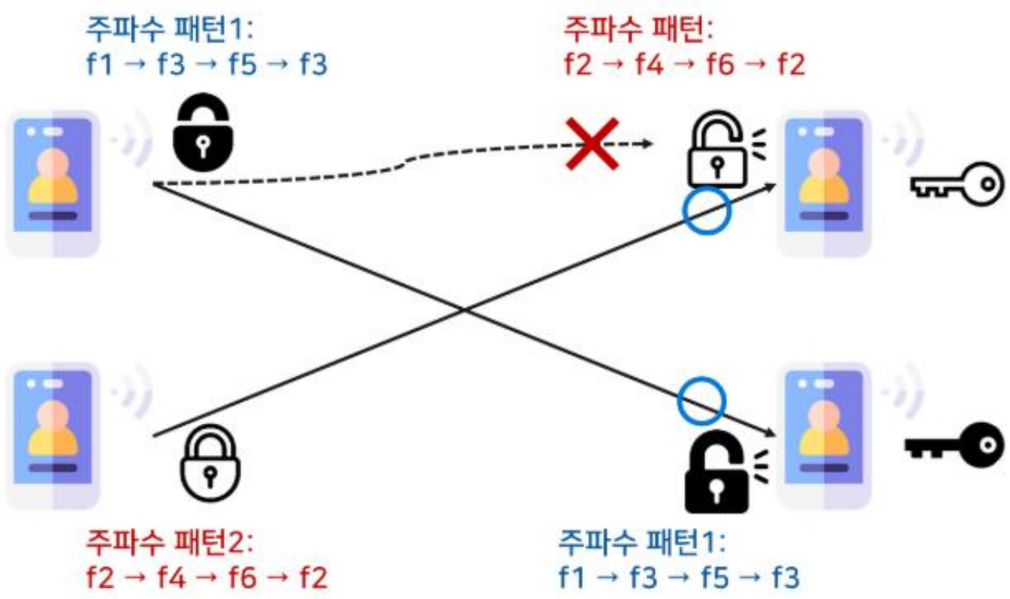
- 데이터를 보낼 때 8비트(1바이트) 데이터를 한 방에 보내는 것이 아니라, 하나의 선을 통해 1비트씩 차례대로 한 줄로 줄 세워 보내기에 배선이 단순해진다는 장점이 있음

비동기(Asynchronous) 통신

- 데이터를 보낼 때 타이밍을 맞춰주는 동기화 시계선(Clock Line)이 없는 대신, 송신축과 수신측이 사전에 Baud Rate(1초에 몇 비트씩 주고받자라는 속도)를 똑같이 약속해 두고 통신을 진행

Bluetooth

요소 기술



Bluetooth

- 블루투스는 2.4GHz ISM 대역을 사용하는 근거리 무선 통신 규격
- 이 주파수 대역은 2.4 ~ 2.48GHz 범위에 있는 79개의 채널을 이용

주파수 호핑(Frequency Hopping Spread Spectrum, FHSS)

- 송신자와 수신자 간의 약속된 주파수 도약 패턴으로, 송신자는 확산된 대역 안에서 그 주파수 패턴대로 정보를 담아 보내고 수신자는 확산된 대역 안에서 약속된 주파수 패턴의 정보만 받아서 암호를 해제하는 방식

PWM

요소 기술

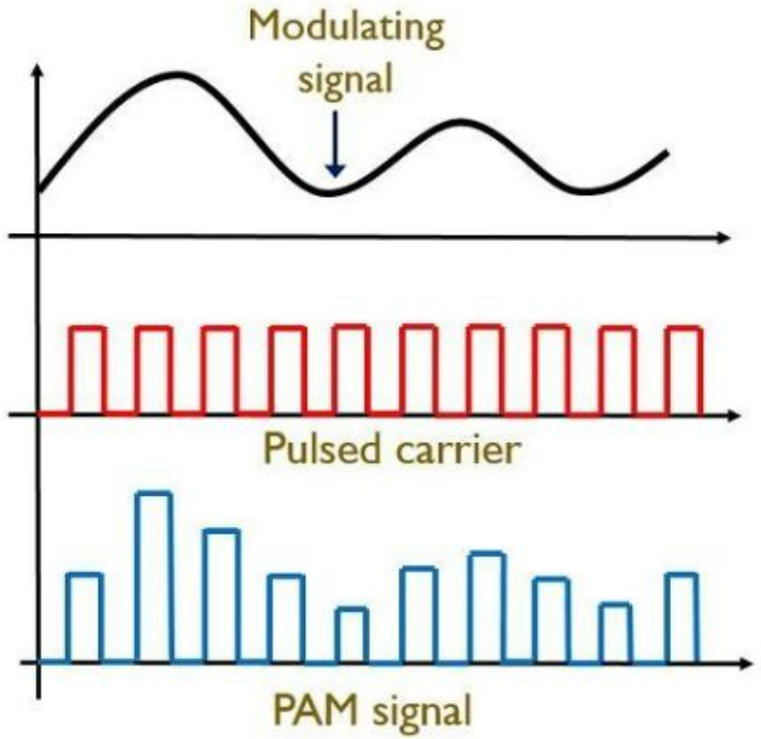


PWM (Pulse Width Modulation, 펄스 폭 변조)

- 원리: 펄스의 높이(전압)는 고정해 두고, 신호가 켜져 있는 너비(폭, Width)를 조절하여 평균 전력을 제어
- 듀티 사이클(Duty Cycle): 전체 주기 중 High(켜진 상태)가 차지하는 비율. 5V 전압에서 듀티 사이클이 50%면 평균 2.5V의 효과를 냄
- 장점: 전압이 항상 일정(최대 아니면 0)하므로 스위칭 손실이 매우 적고 효율이 높으며, 외부 노이즈에 강함
- 주요 용도: DC 모터 속도 제어, LED 밝기 조절(디밍), 인버터 전력 제어

PAM

요소 기술

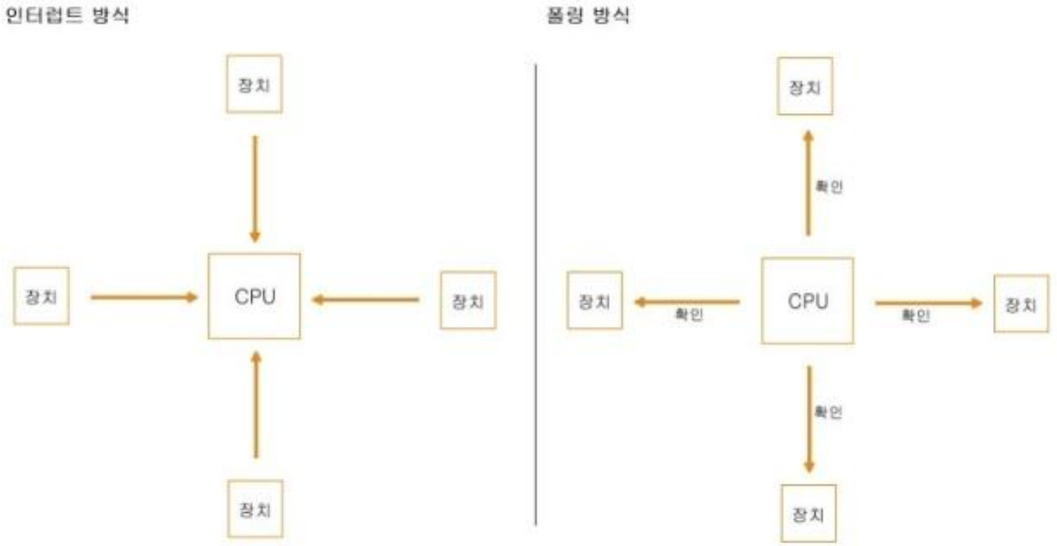


PAM (Pulse Amplitude Modulation, 펄스 진폭 변조)

- 원리: 펄스의 너비는 일정하게 고정해 두고, 원래 아날로그 신호의 크기에 따라 펄스의 높이(진폭, Amplitude)를 변화시킴
- 특징: 입력되는 아날로그 신호의 부드러운 곡선 형태를 계단 모양의 전압 높낮이로 샘플링하여 표현
- 단점: 전압의 높낮이 자체에 데이터가 실리다 보니, 회로적 노이즈나 전압 출렁임이 발생하면 신호가 쉽게 왜곡됨
- 주요 용도: 이더넷(랜선) 통신 신호, 아날로그 신호를 디지털로 변환하는 오디오 데이터의 초기 샘플링 단계

Polling / Interrupt

요소 기술



Polling

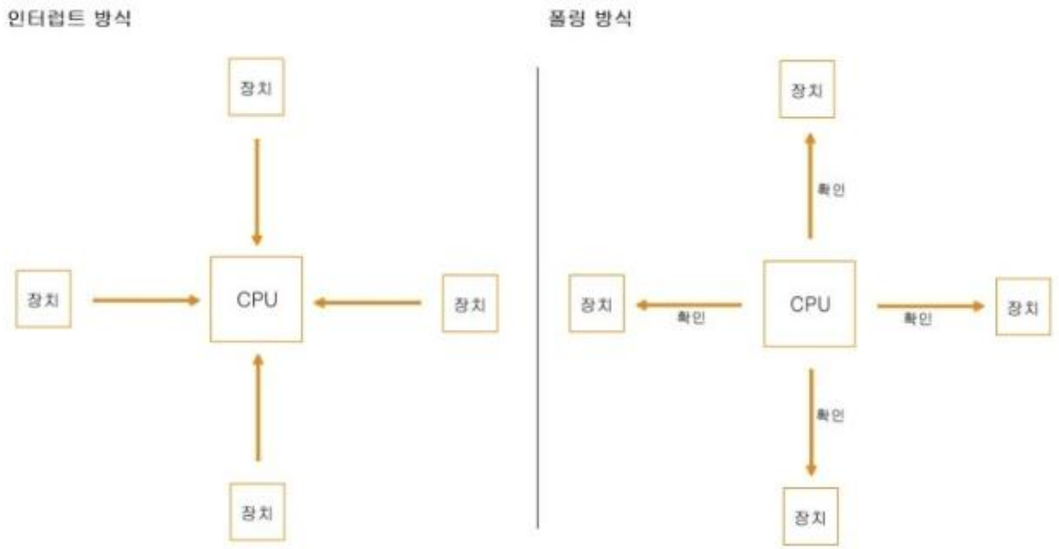
- 정의: cpu가 주기적으로 특정 장치나 상태를 확인하여 이벤트가 발생했는지 직접 처리하는 방식으로,장치 상태를 확인하는 루프를 실행하여 이벤트가 발생했을 때 작업을 처리함

- 장점: 구현이 간단하고 인터럽트 없이 작동이 가능
- cpu가 관여하는 장치들을 일정 주기로 확인하여 신호가 있는지 처리하기 때문에, 각 장치에서 동시에 신호가 발생해도 cpu에서 중복되지 않음 (cpu가 확인하기 전까지는 cpu에게 신호가 들어오지 못함)

- 단점: cpu가 관여할 장치가 많고, 확인하는 주기가 짧을수록 cpu 자원을 낭비함
- 연결된 장치는 신호가 발생해도 cpu가 확인할때 까지 cpu에게 신호를 보내지 못 하기에 이벤트 감지에 지연이 발생할 수 있음

Polling / Interrupt

요소 기술



Interrupt

- 정의: cpu가 명령어 처리 중 외부의 입출력 장치에 대해 인터럽트 신호를 받으면, 해당 인터럽트를 중요도 순으로 처리하는 것

- 장점: 이벤트 시에만 반응하므로 cpu 자원을 효율적으로 처리할 수 있음
- 이벤트 발생 시 즉시 반응하여 실시간 처리가 가능

- 단점: 인터럽트 처리 우선순위 및 동시 처리 문제를 해결해야 함
- 과도한 인터럽트 발생 시 cpu 오버헤드가 증가함

결론



AVR 이해도 향상

ATmega128 데이터 시트를 보며 요구사항들을 구현함으로써 AVR 이해도가 향상



임베디드 SW 기술 이해도 향상

하드웨어 레벨에서의 회로 설계부터 제어까지 자율주행차를 구현함으로써 임베디드 SW 기술 이해도가 향상



데이터 시트의 중요도 체감

초기 회로 설계 과정에서 UART 통신에 사용되는 핀을 FND로 사용했기에 FND가 비정상적으로 동작되는 현상을 체감

결론

- 반드시 데이터 시트를 숙지하고 설계를 해야 한다는 것을 느꼈습니다.
- 단순히 구현만 하는 것이 아닌 기술의 원리를 이해하고 사용해야 결과물의 완성도가 높아진다는 것을 느꼈습니다.

발표를 마치겠습니다

Thank You
감사합니다!

발표자 박상현

010-2791-2479

psh010208@naver